Inventory Structure — Target Design

NextBook ERP

3 August 2026

Reference for the multi-business inventory core. Covers supermarket, grocery, manufacturing, pharmacy, automobile, book, garment, jewellery, mobile, carpet, computer and electric.

Design goal: onboarding a new business type must require **seed data only** — no new tables, no new columns.

---

1. The four levels

Everything below follows from separating these. Mixing them is the root cause of every structural problem in the current schema.

Level	Table	Answers	Carries
Item	`items`	What product line is it?	name, category, accounts, policy
Variant	`item_variants`	Which one does the customer choose?	**SKU, barcode, price, min stock**
Lot	`stock_lots`	Which delivery did it arrive in?	**batch, expiry, MRP, batch cost**
Piece	`stock_pieces`	Which exact physical object?	**serial/IMEI, dimensions, actual cost**

`stock_balances` remains the quantity ledger and points at all four.

Decision rule

Ask in order; first "yes" wins.

1. Does the **customer choose** between them? → **Variant attribute**
2. Is it the same for **every unit** of the product? → **Item field or item attribute**
3. Does it arrive **with a delivery**? → **Lot**
4. Is it unique to **one physical object**? → **Piece**

Rules that must hold

- **Every item has at least one variant.** Items with no choices get a default variant with empty attributes. This makes `variant_id` NOT NULL on all transaction lines and gives barcode scanning and price lookup a single code path.
- **Lots are optional.** No batch and no expiry → no lot row, `lot_id` is null.
- **Pieces are optional.** Only serial/unique-piece items create them.
- For serial-tracked items: `stock_balances.quantity == count(stock_pieces WHERE status = 'in_stock')`

Price resolution

```
lot.sale_price → variant.sale_price → computed (pricing_method)
```

First non-null wins. `items.sale_price` becomes the template that seeds new variants; the variant is authoritative.

---

2. New tables — core hierarchy

2.1 `item_variants`

One row per sellable product.

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();

$table->jsonb('attributes');                        // {"ram":"16GB","color":"black"}
$table->string('variant_key');                      // canonical sorted hash
$table->string('name')->nullable();                 // "16GB / 512GB / Silver"

$table->string('sku')->nullable();
$table->string('barcode')->nullable();

$table->decimal('sale_price', 18, 4)->nullable();
$table->decimal('purchase_price', 18, 4)->nullable();
$table->decimal('avg_cost', 18, 4)->default(0);     // derived, never entered
$table->decimal('minimum_stock', 18, 4)->nullable();
$table->decimal('maximum_stock', 18, 4)->nullable();

$table->decimal('weight', 18, 4)->nullable();
$table->integer('sort_order')->default(0);          // S,M,L sort wrong alphabetically
$table->boolean('is_active')->default(true);

$table->ulid('branch_id')->index();
$table->ulid('created_by')->index();
$table->ulid('updated_by')->nullable();
$table->ulid('deleted_by')->nullable();
$table->timestamps();
$table->softDeletes();
```

```
ALTER TABLE item_variants ADD CONSTRAINT item_variants_identity_unique
  UNIQUE NULLS NOT DISTINCT (item_id, variant_key, deleted_at);
ALTER TABLE item_variants ADD CONSTRAINT item_variants_barcode_unique
  UNIQUE NULLS NOT DISTINCT (branch_id, barcode, deleted_at);
ALTER TABLE item_variants ADD CONSTRAINT item_variants_sku_unique
  UNIQUE NULLS NOT DISTINCT (branch_id, sku, deleted_at);
CREATE INDEX item_variants_attributes_gin ON item_variants USING GIN (attributes);
```

`variant_key` is a deterministic hash of the sorted attribute pairs. Without it nothing prevents two rows both meaning "16GB / Black".

2.2 `stock_lots`

One row per delivery group. **Both identifiers nullable** — an expiry-only lot (milk, bread) is valid and common.

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();
$table->ulid('variant_id')->nullable()->index();

$table->string('batch_no')->nullable();             // null for milk
$table->date('expire_date')->nullable()->index();   // null for paint
$table->date('manufacture_date')->nullable();

$table->decimal('mrp', 18, 4)->nullable();          // printed on the pack
$table->decimal('sale_price', 18, 4)->nullable();   // what you charge for this lot
$table->decimal('unit_cost', 18, 4)->nullable();    // real cost of THIS lot
$table->decimal('purchase_price', 18, 4)->nullable();

$table->ulid('supplier_id')->nullable()->index();
```

```
$table->date('received_date')->nullable();
$table->string('status')->default('active');        // active|blocked|recalled|expired

$table->ulid('branch_id')->index();
// + audit columns, timestamps, softDeletes
```

```
ALTER TABLE stock_lots ADD CONSTRAINT stock_lots_identity_unique
  UNIQUE NULLS NOT DISTINCT
  (branch_id, item_id, variant_id, batch_no, expire_date, deleted_at);
```

`NULLS NOT DISTINCT` (PostgreSQL 15+) is **required** here. Without it, two rows with `batch_no = NULL` and the same expiry do not conflict, and you get duplicate lots each with their own price.

Display label: `batch_no ?? expire_date->format(...)` — a pharmacist sees "B-1043", a baker sees "Exp 12/08/2026".

2.3 `stock_pieces`

One row per physical object. Named *pieces*, not *units*, to avoid collision with unit-of-measure.

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();
$table->ulid('variant_id')->index();
$table->ulid('lot_id')->nullable()->index();
$table->ulid('warehouse_id')->index();

$table->jsonb('identifiers');    // {"imei1":"…","imei2":"…","vin":"…","hallmark":"…"}
$table->jsonb('attributes');     // {"length":2.1,"width":3.0,"net_weight":8.4}
$table->decimal('measured_quantity', 18, 4)->nullable();  // 6.3 m² / 8.4 g

$table->decimal('unit_cost', 18, 4)->nullable();    // specific identification
$table->decimal('sale_price', 18, 4)->nullable();
$table->string('condition')->nullable();            // new|refurbished|used|damaged
$table->string('status')->default('in_stock');      // in_stock|reserved|sold|returned|scrapped

$table->ulid('purchase_item_id')->nullable();
$table->ulid('sale_item_id')->nullable();
$table->date('warranty_end_date')->nullable();

$table->ulid('branch_id')->index();
// + audit columns, timestamps, softDeletes
```

`identifiers` is JSONB, not a `serial_no` column: dual-SIM phones need **two** IMEIs, vehicles need VIN *and* engine number, carpets need neither.

---

3. New tables — attribute engine

Business-defined. Shipping a new business type adds **rows, not columns**.

3.1 `attributes`

```
$table->ulid('id')->primary();
$table->string('code')->index();            // ram, color, purity, voltage
$table->string('name');
$table->string('input_type');               // select|text|number|boolean|date
$table->string('unit')->nullable();         // GB, mm, gram
```

```
$table->boolean('is_variant_forming')->default(false);
$table->integer('sort_order')->default(0);
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
```

`is_variant_forming` is the switch that does the work — and the same attribute differs by business:

Attribute	Garment	Book	Computer
Colour	**variant**	descriptive	**variant**
Size	**variant**	descriptive	n/a
Storage	n/a	n/a	**variant**

3.2 `attribute_values`

```
$table->ulid('id')->primary();
$table->ulid('attribute_id')->index();
$table->string('value');
$table->string('label')->nullable();
$table->string('swatch_hex')->nullable();   // colour picker
$table->integer('sort_order')->default(0);
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
```

3.3 `category_attributes`

```
$table->ulid('id')->primary();
$table->ulid('category_id')->index();
$table->ulid('attribute_id')->index();
$table->boolean('is_required')->default(false);
$table->unique(['category_id', 'attribute_id']);
```

---

4. New tables — item extensions

4.1 `item_units` — per-item UoM conversion

Fixes an active correctness bug: conversion factors currently come from the global `unit_measures.unit`, so "box" is always 6 regardless of item.

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();
$table->ulid('unit_measure_id')->index();
$table->decimal('conversion_factor', 18, 6);   // base units per 1 of this unit
$table->string('barcode')->nullable();         // scan the carton vs the piece
$table->boolean('is_base')->default(false);
$table->boolean('is_purchase_default')->default(false);
$table->boolean('is_sale_default')->default(false);
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
$table->unique(['item_id', 'unit_measure_id', 'deleted_at']);
```

4.2 `item_barcodes` — many codes per variant

```
$table->ulid('id')->primary();
```

```
$table->ulid('item_id')->index();
$table->ulid('variant_id')->nullable()->index();
$table->ulid('unit_measure_id')->nullable();   // which pack size this code is for
$table->string('barcode');
$table->string('type')->default('gtin');       // gtin|ean13|upc|supplier|internal
$table->boolean('is_primary')->default(false);
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
$table->unique(['branch_id', 'barcode', 'deleted_at']);
```

4.3 `item_suppliers`

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();
$table->ulid('variant_id')->nullable()->index();
$table->ulid('supplier_id')->index();
$table->string('supplier_item_code')->nullable();
$table->decimal('last_price', 18, 4)->nullable();
$table->date('last_purchase_date')->nullable();
$table->integer('lead_time_days')->nullable();
$table->decimal('min_order_qty', 18, 4)->nullable();
$table->boolean('is_preferred')->default(false);
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
```

Maintained automatically on purchase posting.

4.4 `item_relations`

Pharmacy generics, interchangeable auto parts, accessories, cross-sell.

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();
$table->ulid('related_item_id')->index();
$table->string('type');    // substitute|accessory|spare_part|cross_sell|upsell
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
$table->unique(['item_id', 'related_item_id', 'type', 'deleted_at']);
```

4.5 `item_components` — kits and BOM

PC assembly, combo packs, manufacturing.

```
$table->ulid('id')->primary();
$table->ulid('parent_item_id')->index();
$table->ulid('parent_variant_id')->nullable()->index();
$table->ulid('component_item_id')->index();
$table->ulid('component_variant_id')->nullable()->index();
$table->decimal('quantity', 18, 4);
$table->ulid('unit_measure_id')->nullable();
$table->decimal('scrap_percent', 8, 4)->default(0);
$table->string('type')->default('kit');    // kit|bom
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
```

4.6 `item_fitments` — compatibility

Automobile parts, computer components, mobile accessories.

```
$table->ulid('id')->primary();
$table->ulid('item_id')->index();
```

```
$table->jsonb('criteria');   // {"make":"Toyota","model":"Corolla","year_from":2015,"year_to":2020}
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
```

```
CREATE INDEX item_fitments_criteria_gin ON item_fitments USING GIN (criteria);
```

---

5. New tables — pricing and tax

5.1 `market_rates` — computed pricing

Jewellery (gold rate), carpet (rate per m²), electric (rate per metre). Same shape as your existing `currency_rate_updates`.

```
$table->ulid('id')->primary();
$table->string('rate_type')->index();      // gold_22k|gold_18k|silver|carpet_m2
$table->decimal('rate', 18, 4);
$table->ulid('currency_id')->nullable();
$table->date('date')->index();
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
$table->unique(['branch_id', 'rate_type', 'date', 'deleted_at']);
```

5.2 `tax_rates`

Currently absent entirely — `sale_items.tax` and `purchase_items.tax` are bare decimals typed in by hand with no GL posting.

```
$table->ulid('id')->primary();
$table->string('name');
$table->decimal('rate', 8, 4);
$table->boolean('is_inclusive')->default(false);
$table->ulid('sale_tax_account_id')->nullable();
$table->ulid('purchase_tax_account_id')->nullable();
$table->boolean('is_active')->default(true);
$table->ulid('branch_id')->index();
// + audit, timestamps, softDeletes
```

5.3 `price_lists` / `price_list_items`

Replaces the unnamed `rate_a` / `rate_b` / `rate_c` columns, which have no mapping to `customer_groups` today.

```
// price_lists
$table->ulid('id')->primary();
$table->string('name');
$table->ulid('currency_id')->nullable();
$table->ulid('customer_group_id')->nullable()->index();
$table->date('starts_at')->nullable();
$table->date('ends_at')->nullable();
$table->integer('priority')->default(0);
$table->boolean('is_active')->default(true);
$table->ulid('branch_id')->index();

// price_list_items
$table->ulid('id')->primary();
$table->ulid('price_list_id')->index();
$table->ulid('item_id')->index();
```

```
$table->ulid('variant_id')->nullable()->index();
$table->decimal('min_quantity', 18, 4)->default(0);   // quantity breaks
$table->decimal('price', 18, 4);
$table->decimal('discount_percent', 8, 4)->nullable();
$table->ulid('branch_id')->index();
```

---

6. Modified tables

6.1 `items`

Add — behaviour flags (a business type is a *combination of flags*):

```
$table->boolean('is_active')->default(true);
$table->boolean('is_stockable')->default(true);      // false for services
$table->boolean('is_sellable')->default(true);       // raw materials: false
$table->boolean('is_purchasable')->default(true);
$table->boolean('is_serial_tracked')->default(false);
$table->boolean('is_weighted')->default(false);      // supermarket produce
$table->boolean('has_variants')->default(false);
$table->boolean('allow_negative_stock')->default(false);
```

Add — pricing and costing:

```
$table->string('pricing_method')->default('fixed');
    // fixed|weight_based|area_based|length_based|rate_based
$table->string('costing_method')->nullable();        // null = company default
$table->string('rate_type')->nullable();             // → market_rates
$table->decimal('making_charge', 18, 4)->nullable();
$table->string('making_charge_type')->nullable();    // fixed|percent|per_gram
$table->decimal('wastage_percent', 8, 4)->nullable();
$table->string('ownership_type')->default('owned');  // owned|consignment_in|consignment_out
```

Add — physical and regulatory:

```
$table->text('description')->nullable();
$table->decimal('weight', 18, 4)->nullable();
$table->decimal('length', 18, 4)->nullable();
$table->decimal('width', 18, 4)->nullable();
$table->decimal('height', 18, 4)->nullable();
$table->string('manufacturer')->nullable();
$table->string('country_of_origin')->nullable();
$table->string('hs_code')->nullable();
$table->integer('warranty_months')->nullable();
$table->integer('shelf_life_days')->nullable();
$table->integer('min_shelf_life_percent')->nullable();  // reject on receipt below this
$table->string('storage_zone')->nullable();             // ambient|chilled|frozen
$table->boolean('requires_prescription')->default(false);
$table->boolean('is_controlled')->default(false);
```

Add — planning and tax:

```
$table->decimal('reorder_quantity', 18, 4)->nullable();
$table->integer('lead_time_days')->nullable();
$table->ulid('default_warehouse_id')->nullable();
$table->ulid('sale_tax_rate_id')->nullable();
$table->ulid('purchase_tax_rate_id')->nullable();
$table->boolean('is_tax_exempt')->default(false);
```

```
$table->string('abc_class')->nullable();
$table->jsonb('attributes')->nullable();            // descriptive, non-variant
```

Add — manufacturing hooks (reserve now, module later):

```
$table->ulid('default_bom_id')->nullable();
$table->decimal('scrap_percent', 8, 4)->nullable();
$table->integer('production_lead_time')->nullable();
```

Keep unchanged: `is_batch_tracked` and `is_expiry_tracked` stay as two independent booleans. They are **not** collapsible into one enum — a supermarket needs "expiry but no batch" for milk and bread.

Remove after backfill:

- `colors` (json) → variant attributes
- `size_id` → variant attributes
- `is_color_tracked`, `is_size_tracked` → replaced by the attribute engine
- `rate_a`, `rate_b`, `rate_c` → `price_lists` (retain during transition)

Fix:

- `photo` is in `$fillable` and assigned in `ItemController::update()` but **the column does not exist** — add it, or use `attachments` with `is_primary`
- Replace `unique(['branch_id','name','deleted_at'])` with `UNIQUE NULLS NOT DISTINCT` — the current constraint never fires on live rows because `deleted_at` is NULL (see §8)

6.2 `stock_balances`

```
// add
$table->ulid('variant_id')->nullable()->index();
$table->ulid('lot_id')->nullable()->index();
$table->string('ownership_type')->default('owned');

// remove after backfill
// batch, expire_date  → stock_lots
// color, size_id      → item_variants
```

New grain: `(branch, item, variant, lot, warehouse, ownership)`.

Keep `quantity`, `reserved_out`, `reserved_in`, `status` unchanged.

Consignment-in stock must be excluded from inventory valuation — you hold it but do not own it. Including it overstates assets on the balance sheet.

6.3 `stock_movements`

Same additions as `stock_balances`, plus `piece_id` nullable for serial items.

6.4 Transaction line tables (13)

`purchase_items`, `purchase_return_items`, `purchase_order_items`, `purchase_quotation_items`, `sale_items`, `sale_return_items`, `sale_order_items`, `sale_quotation_items`, `item_transfer_items`, `stock_adjustment_items`, `landed_cost_items`, `stock_outs`, `stock_movements`:

```
$table->ulid('variant_id')->nullable()->index();   // NOT NULL after backfill
$table->ulid('lot_id')->nullable()->index();
$table->ulid('piece_id')->nullable()->index();
// remove after backfill: batch, expire_date, color, size_id
```

Note: six of these currently have `size_id` but **no `color`** — `stock_outs`, `purchase_return_items`, `purchase_order_items`, `purchase_quotation_items`, `sale_quotation_items`, `item_transfer_items`. Do **not** add `color` to them. Go straight to `variant_id`.

6.5 `categories` / `warehouses`

```
// categories — is_active is in $fillable and $casts but the column doesn't exist
$table->boolean('is_active')->default(true);
$table->ulid('default_asset_account_id')->nullable();   // cascade to new items
$table->ulid('default_income_account_id')->nullable();
$table->ulid('default_cost_account_id')->nullable();
$table->decimal('default_margin_percent', 8, 4)->nullable();
$table->ulid('default_tax_rate_id')->nullable();

// warehouses
$table->string('zone_type')->nullable();    // ambient|chilled|frozen
```

6.6 `users` → `companies`

Move `preferences` from `users` to `companies`. A business type is a property of the company, not of each employee. Today two staff at the same shop can have different item forms, and a new hire gets hardcoded defaults instead of the company's shape. Keep user-level overrides layered on top if wanted.

---

7. Enum changes

Enum	Change
`ItemType`	add `SEMI_FINISHED`, `WIP`
`CostingMethod`	add `SPECIFIC` (jewellery, carpet, vehicles), `STANDARD` (manufacturing)
`StockSourceType`	add `PRODUCTION_ISSUE`, `PRODUCTION_RECEIPT`, `REPACK`
`BusinessType`	add `MANUFACTURING`, `COMPUTER`, `ELECTRIC` — and **wire it to seed data**; today it is stored and validated but drives nothing

New enums: `PricingMethod`, `OwnershipType`, `PieceStatus`, `LotStatus`, `AttributeInputType`, `ItemCondition`.

`costing_method` moves from company-wide (`Cache::get('costing_method')`) to per-item, falling back to the company default. A jewellery shop sells gold (specific) *and* gift boxes (weighted average).

---

8. Cross-cutting fixes required

PostgreSQL NULL uniqueness

Every table using `unique([..., 'deleted_at'])` has a **non-functioning constraint** — PostgreSQL treats NULLs as distinct, and `deleted_at` is NULL on every live row. Duplicate names and codes are currently possible on `items`, `categories`, `brands`, `warehouses`, `sizes`, `unit_measures`.

```
ALTER TABLE items ADD CONSTRAINT items_name_unique
```

```
    UNIQUE NULLS NOT DISTINCT (branch_id, name, deleted_at);
```

PG 15+; you are on 16. Laravel's schema builder cannot express this — use `DB::statement()` in the migration.

Validation rules

`ItemStoreRequest` uses `unique:items,name,NULL,id,branch_id,NULL,...`, which tests `branch_id IS NULL` and never matches. Combined with the above, duplicates escape **both** layers.

Unit conversion

Replace the three duplicated conversion implementations (`StockService::resolveConversionFactor`, and private copies in `PurchaseController` and `SaleController`) with one service reading `item_units`.

The average-cost recompute in `PurchaseController` and `SaleController` reads `stock_movements.quantity` **with no unit conversion at all** — buy 1 box at 600 and `avg_cost` becomes 600 per box, stored and read everywhere as per-piece.

---

9. Business coverage

Business	Variant attributes	Lot	Piece	Pricing	Costing
Supermarket	flavour, pack	**batch + expiry**	—	fixed / weight	average
Grocery	pack	**expiry**	—	fixed / weight	average
Manufacturing	per product	batch	sometimes	fixed	standard + BOM
Pharmacy	strength, form	**batch + expiry**	—	fixed	FIFO
Automobile — parts	model, position	—	sometimes	fixed	average
Automobile — vehicles	model, year	—	**VIN**	fixed	**specific**
Book	edition, format	—	—	fixed	average
Garment	**size, colour**	—	—	fixed	average
Jewellery	purity, design	—	**hallmark**	**weight × rate**	**specific**
Mobile	RAM, storage, colour	—	**IMEI ×2**	fixed	**specific**
Carpet	design, origin	—	**per piece**	**area × rate**	**specific**
Computer	RAM, storage, CPU	—	**serial**	fixed	specific / FIFO
Electric	voltage, gauge	—	sometimes	**length × rate**	average

Same tables throughout. Only rows and flags differ.

---

10. Migration phases

Phase	Scope	Effort
0	Working `ItemTest` (current file is a generated stub referencing 4 non-existent model classes); fix the P0 bugs	3 days
A	Behaviour flags, item fields, `item_units`, preferences → company, NULL-uniqueness fixes	1 week
B	Attribute engine, `business_type` seeders	1.5 weeks

Phase	Scope	Effort
C	`item_variants` + `variant_id` across 13 tables + backfill	2 weeks
D	`stock_lots` + `lot_id` + price resolution *(supermarket-ready)*	1 week
E	`stock_pieces` + specific costing	1 week
F	`market_rates`, computed pricing *(unblocks jewellery, carpet)*	1 week
G	Tax, price lists, suppliers, relations, barcodes, fitments	1.5 weeks
H	Manufacturing module — BOM, production orders, WIP, routing	3–4 weeks

Core (0–G): ~9 weeks. Phase H is separate.

Phases **C, D and E** must not be postponed — their cost scales with transaction volume, not code size. Backfill is only possible while `stock_movements` history is intact.

Suggested proving order

1. **Supermarket** — phases A, B, D. Validates lots, expiry-only lots, FEFO.
2. **Jewellery** — adds E and F. Exercises computed pricing, specific costing and per-piece tracking simultaneously. If the schema survives jewellery, the rest are straightforward.
3. Everything else.

---

11. Open decisions

1. **Are `stock_lots` per branch or shared across branches?** Per branch matches your existing scoping, but the same physical delivery split between two shops becomes two rows with two prices.
2. **Do lot-price corrections post a GL adjustment** through `StockAdjustment`, or rewrite balances silently? Adjustment is the auditable answer.
3. **Repair history, or forward-only?** Full repair is possible now because `stock_movements` is intact; it stops being possible once old movements are archived.

---

12. What already works — do not change

- **FEFO allocation.** `StockService::deductFIFO` orders by `expire_date` with nulls last, then date. Correct for supermarket and pharmacy, and it already handles the mixed expiry/non-expiry case.
- **Reservations.** `reserved_out` / `reserved_in` on `stock_balances` prevent overselling from unposted lines.
- **Batch on-hand aggregation** in `SearchController`. Note the per-batch `avg_cost` it returns is the **item's** average copied onto each batch — that becomes real once `stock_lots.unit_cost` exists.
- `spec_text` in preferences already relabels "Batch" per business.
- **Polymorphic `attachments`** — extend with `collection` and `is_primary` for images rather than adding a `photo` column.